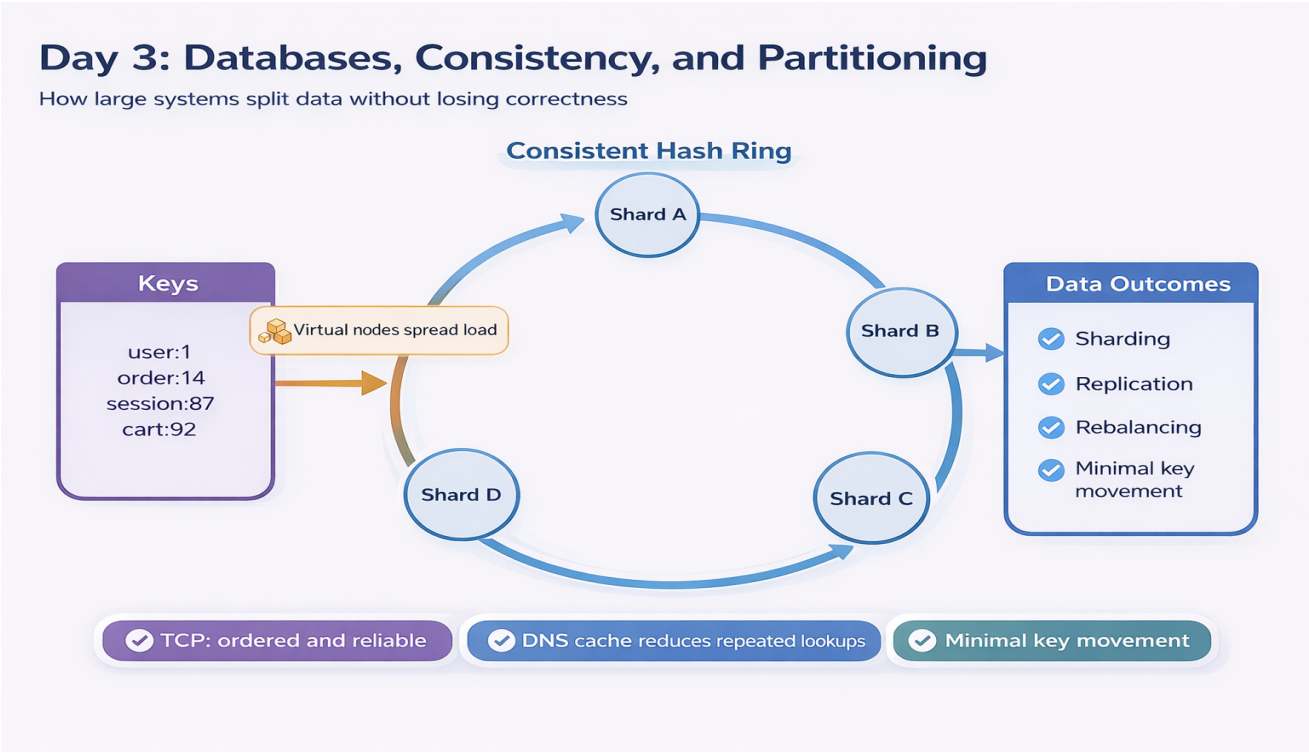
Day 3: Databases, Consistency, and Partitioning

Day 3 focuses on how large systems store, protect, and distribute data.

It covers:

- databases and DBMS fundamentals
- SQL vs NoSQL trade-offs
- replication and indexes
- normalization and denormalization
- ACID, BASE, CAP, and PACELC
- transactions and distributed transactions
- sharding, consistent hashing, and federation

In short, Day 3 is about choosing the right data model and distribution strategy without ignoring consistency and operational cost.



Core ideas

Databases and DBMS

- A `database` stores structured or semi-structured data.
- A `DBMS` manages access, schema, indexing, durability, backup, and recovery.
- Tables, rows, columns, and schema matter even when the final system becomes distributed.

SQL vs NoSQL

- `SQL` is strong for joins, transactions, and rigid schema.
- `NoSQL` is strong for flexible schema, scale-out workloads, and high write/read throughput.
- The right choice depends on workload, query shape, and consistency requirements.

Replication and indexes

- Replication improves availability and read scaling, but introduces lag or conflict handling.
- Indexes improve read performance but increase write amplification and storage cost.

Normalization vs denormalization

- `Normalization` reduces redundancy and improves consistency.
- `Denormalization` duplicates data to reduce joins and improve read performance.
- Most production systems use a balance of both.

Consistency models

- `ACID` favors correctness and transactional guarantees.
- `BASE` favors availability and eventual convergence at scale.
- `CAP` explains the partition-time trade-off between consistency and availability.
- `PACELC` adds the normal-case latency vs consistency trade-off.

Transactions and partitioning

- Single-node transactions are simpler.
- Distributed transactions need coordination like `2PC`, `3PC`, or sagas.
- `Sharding` spreads data across machines.
- `Consistent hashing` reduces rebalancing cost when shards are added or removed.

Day-3 sample: Consistent Hashing Shard Router

This repository includes a small consistent hashing sample in both Python and Java.

Functional requirements

- Add logical shard nodes to a hash ring
- Route a key to its owning shard
- Support virtual nodes for more even distribution
- Show how mappings change when a new shard is added

High-level components

- `Client/API`: produces keys such as user IDs or order IDs
- `Hash ring`: maps keys to points on a ring
- `Shard nodes`: store the actual data partitions
- `Virtual nodes`: improve load distribution

Data flow

1. Each shard is placed on the hash ring using multiple virtual nodes
2. A key is hashed into a ring position
3. The system walks clockwise to the next shard
4. That shard becomes the owner for the key
5. When a new shard is added, only a subset of keys move

Project structure

```
Day-3/
  README.md
  consistent-hashing-sample/
    python/
      consistent_hashing.py
      demo.py
    java/
      ConsistentHashRing.java
      Main.java
```

Run the sample

Python

```
cd consistent-hashing-sample\python
python demo.py
```

Java

```
cd consistent-hashing-sample\java
javac Main.java ConsistentHashRing.java
java Main
```

Interview takeaways

- SQL vs NoSQL is a workload decision, not a religion
- Replication improves resilience, but it changes failure behavior
- Sharding helps scale, but it complicates joins and rebalancing
- Consistent hashing is useful when nodes are expected to come and go
- CAP is a constraint, not an architecture recipe

Next improvements

- Add replication factor to the hash ring
- Simulate read replicas and failover
- Add hot-key detection
- Compare simple modulo hashing vs consistent hashing
- Extend the sample into a sharded key-value store

Source Code Appendix

Day-3/consistent-hashing-sample/python/consistent_hashing.py

```python
from __future__ import annotations

from bisect import bisect_right
from dataclasses import dataclass, field
import hashlib


@dataclass
class ConsistentHashRing:
    virtual_nodes: int = 10
    _ring: list[int] = field(default_factory=list, init=False)
    _owners: dict[int, str] = field(default_factory=dict, init=False)

    def add_node(self, node: str) -> None:
        for replica in range(self.virtual_nodes):
            key = self._hash(f"{node}#{replica}")
            if key not in self._owners:
                self._owners[key] = node
                self._ring.append(key)
        self._ring.sort()

    def get_node(self, item_key: str) -> str:
        if not self._ring:
            raise RuntimeError("Hash ring is empty")

        key = self._hash(item_key)
        index = bisect_right(self._ring, key)
        if index == len(self._ring):
            index = 0
        return self._owners[self._ring[index]]

    def distribution(self, keys: list[str]) -> dict[str, int]:
        result: dict[str, int] = {}
        for key in keys:
            owner = self.get_node(key)
            result[owner] = result.get(owner, 0) + 1
        return dict(sorted(result.items()))

    @staticmethod
    def _hash(value: str) -> int:
        digest = hashlib.sha256(value.encode("utf-8")).hexdigest()
        return int(digest[:8], 16)
```

Day-3/consistent-hashing-sample/python/demo.py

```python
from consistent_hashing import ConsistentHashRing


def main() -> None:
    keys = [f"user:{i}" for i in range(1, 21)]

    ring = ConsistentHashRing(virtual_nodes=20)
    ring.add_node("shard-a")
    ring.add_node("shard-b")
    ring.add_node("shard-c")

    print("Initial routing:")
    for key in keys[:8]:
        print(f"  {key} -> {ring.get_node(key)}")
    print("Distribution:", ring.distribution(keys))

    ring.add_node("shard-d")
    print("\nAfter adding shard-d:")
    for key in keys[:8]:
        print(f"  {key} -> {ring.get_node(key)}")
    print("Distribution:", ring.distribution(keys))


if __name__ == "__main__":
    main()
```

Day-3/consistent-hashing-sample/java/ConsistentHashRing.java

```java
import java.nio.charset.StandardCharsets;
import java.security.MessageDigest;
import java.security.NoSuchAlgorithmException;
import java.util.ArrayList;
```

```java
import java.util.Collections;
import java.util.HashMap;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;

public class ConsistentHashRing {
    private final int virtualNodes;
    private final List<Integer> ring;
    private final Map<Integer, String> owners;

    public ConsistentHashRing(int virtualNodes) {
        this.virtualNodes = virtualNodes;
        this.ring = new ArrayList<>();
        this.owners = new HashMap<>();
    }

    public synchronized void addNode(String node) {
        for (int replica = 0; replica < virtualNodes; replica++) {
            int key = hash(node + "#" + replica);
            if (!owners.containsKey(key)) {
                owners.put(key, node);
                ring.add(key);
            }
        }
        Collections.sort(ring);
    }

    public synchronized String getNode(String itemKey) {
        if (ring.isEmpty()) {
            throw new IllegalStateException("Hash ring is empty");
        }

        int key = hash(itemKey);
        int index = Collections.binarySearch(ring, key);
        if (index < 0) {
            index = -index - 1;
        }
        if (index == ring.size()) {
            index = 0;
        }
        return owners.get(ring.get(index));
    }

    public synchronized Map<String, Integer> distribution(List<String> keys) {
        Map<String, Integer> counts = new HashMap<>();
        for (String key : keys) {
            String owner = getNode(key);
            counts.put(owner, counts.getOrDefault(owner, 0) + 1);
        }

        List<String> sortedKeys = new ArrayList<>(counts.keySet());
        Collections.sort(sortedKeys);
        Map<String, Integer> ordered = new LinkedHashMap<>();
        for (String key : sortedKeys) {
            ordered.put(key, counts.get(key));
        }
        return ordered;
    }

    private static int hash(String value) {
        try {
            MessageDigest digest = MessageDigest.getInstance("SHA-256");
            byte[] bytes = digest.digest(value.getBytes(StandardCharsets.UTF_8));
            return ((bytes[0] & 0xff) << 24)
                    | ((bytes[1] & 0xff) << 16)
                    | ((bytes[2] & 0xff) << 8)
                    | (bytes[3] & 0xff);
        } catch (NoSuchAlgorithmException e) {
            throw new IllegalStateException("SHA-256 is not available", e);
        }
    }
}
```

Day-3/consistent-hashing-sample/java/Main.java

```java
import java.util.ArrayList;
import java.util.List;

public class Main {
    public static void main(String[] args) {
        List<String> keys = new ArrayList<>();
        for (int i = 1; i <= 20; i++) {
            keys.add("user:" + i);
```

```
        }

        ConsistentHashRing ring = new ConsistentHashRing(20);
        ring.addNode("shard-a");
        ring.addNode("shard-b");
        ring.addNode("shard-c");

        System.out.println("Initial routing:");
        for (int i = 0; i < 8; i++) {
            String key = keys.get(i);
            System.out.println("  " + key + " -> " + ring.getNode(key));
        }
        System.out.println("Distribution: " + ring.distribution(keys));

        ring.addNode("shard-d");
        System.out.println();
        System.out.println("After adding shard-d:");
        for (int i = 0; i < 8; i++) {
            String key = keys.get(i);
            System.out.println("  " + key + " -> " + ring.getNode(key));
        }
        System.out.println("Distribution: " + ring.distribution(keys));
    }
}
```